🗄️ AWS Databases — Category 6: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** RDS, DynamoDB, Aurora, ElastiCache, RDS Proxy, Redshift, Parameter Groups, Snapshots, DAX, Streams, TTL, Hot Partitions, Distribution Styles

---

📋 Table of Contents

1. 6.1 RDS — Engines, Multi-AZ, Read Replicas
2. 6.2 DynamoDB — Tables, Items, Partition Keys, GSI/LSI
3. 6.3 RDS — Parameter Groups, Option Groups, Automated Backups, Snapshots
4. 6.4 Aurora — Architecture, Aurora Serverless, Global Database
5. 6.5 DynamoDB — Capacity Modes, DAX, Streams, TTL
6. 6.6 ElastiCache — Redis vs Memcached, Cluster Mode
7. 6.7 Aurora — Storage Architecture, Fast Failover, Parallel Query
8. 6.8 DynamoDB — Hot Partition Problem, Adaptive Capacity, Design Patterns
9. 6.9 RDS Proxy — Connection Pooling, Use with Lambda
10. 6.10 Redshift — Architecture, Distribution Styles, Sort Keys

---

6.1 RDS — Engines, Multi-AZ, Read Replicas

🟢 What It Is in Simple Terms

RDS (Relational Database Service) is AWS's managed relational database service. You pick your engine (MySQL, PostgreSQL, Oracle, SQL Server, MariaDB), and AWS handles the undifferentiated heavy lifting: OS patching, backups, hardware provisioning, replication, and failover.

🔍 Why It Exists / What Problem It Solves

Running a production database on EC2 means you manage everything — OS, database installation, replication setup, backup scripts, monitoring, patching, and failover automation. RDS removes all of that operational burden so your team focuses on schema design and query optimization, not database administration.

⚙️ How It Works Internally

```
RDS Architecture:

┌──────────────────────────────────────────────────────────┐
│                      AWS Account                          │
│                                                           │
│   ┌──────────────────────┐     ┌──────────────────────┐   │
│   │  Primary DB          │     │  Standby DB          │   │
│   │  (AZ-a)              │     │  (AZ-b)              │   │
│   │  Serving reads       │     │  Synchronous replica │   │
│   │  & writes            │     │  NOT serving traffic │   │
│   └──────────────────────┘     └──────────────────────┘   │
│            │                                              │
│            │                                              │
│            │ synchronous replication (Multi-AZ)           │
│            └──────────────────────────────────────────────┘
│
│  DNS: mydb.xyz.us-east-1.rds.amazonaws.com
│       (always points to current primary)
└──────────────────────────────────────────────────────────┘

Your app always connects to the DNS endpoint.
On failover: DNS flips to standby (now primary) automatically.
```

🧩 RDS Engines

```
Supported engines:
├── MySQL (5.7, 8.0)
├── PostgreSQL (13, 14, 15, 16)
├── MariaDB (10.5, 10.6, 10.11)
├── Oracle (SE2, EE — BYOL or license-included)
├── SQL Server (SE, EE, Web, Express)
└── (Aurora is a separate service — different architecture)

Engine selection criteria:
├── MySQL / MariaDB: web apps, LAMP stack, open source
├── PostgreSQL: complex queries, JSONB, extensions, PostGIS
├── Oracle: legacy enterprise apps, specific Oracle features
└── SQL Server: .NET apps, Windows-centric organizations

RDS instance classes:
├── db.t3.* / db.t4g.*: burstable (dev/test)
├── db.m5.* / db.m6g.*: general purpose (production)
├── db.r5.* / db.r6g.*: memory optimized (large DBs, caching)
└── db.x2g.*:           extreme memory (large in-memory DBs)

Storage types:
├── gp3: general purpose SSD (default) — 3,000 IOPS baseline
├── io1/io2: provisioned IOPS SSD — up to 64,000 IOPS
└── magnetic: legacy, avoid for new instances
```

🧩 Multi-AZ

```
Multi-AZ = synchronous standby in a different AZ
Purpose: HIGH AVAILABILITY — not performance improvement

How it works:
├── Primary instance serves all reads and writes
├── Every write is synchronously replicated to standby BEFORE
│   acknowledging the write to the application
│   (write latency is slightly higher because of this)
├── Standby does NOT serve any traffic
│   (you cannot read from the standby)
└── On failover: DNS CNAME flips to standby (60-120 sec)

Failover triggers:
├── AZ failure
├── Primary host hardware failure
├── Primary OS failure
├── DB software crash
├── Manual failover (reboot with failover option)
└── DB instance type change (causes planned failover)

Failover behavior:
1. RDS detects primary failure
2. Promotes standby to primary
3. Updates DNS record (CNAME flip)
4. Old DNS TTL: 5 seconds (keep short in app config!)
5. Application reconnects to new primary via same endpoint
Total failover time: typically 60-120 seconds

⚠ Multi-AZ is SYNCHRONOUS replication
   Every write waits for confirmation from standby.
   This adds ~1-5ms latency to writes.
   Worth it for production — prevents data loss.

⚠ Multi-AZ standby is NOT a read replica.
   You CANNOT send SELECT queries to standby.
   Use Read Replicas for read scaling.

Multi-AZ modes:
├── Single standby: 1 standby in 1 AZ (standard Multi-AZ)
└── Multi-AZ Cluster: 1 writer + 2 readable standbys (newer,
PostgreSQL/MySQL)
    Readable standbys use semi-synchronous replication
    Recovery time < 35 seconds
```

```
# Enable Multi-AZ on existing instance
aws rds modify-db-instance \
  --db-instance-identifier mydb \
```

```
  --multi-az \
  --apply-immediately
```

🧩 Read Replicas

```
Read Replicas = ASYNCHRONOUS copies for READ SCALING
Purpose: PERFORMANCE — offload read traffic from primary

How it works:
├── Data written to primary
├── Replication log sent ASYNCHRONOUSLY to read replica
├── Replica applies changes with slight lag (replication lag)
├── Read replicas are readable — you connect to their endpoint
└── Replicas have their OWN endpoint (not same as primary)

Your app must explicitly direct reads to replica endpoint.

Read Replica limits:
├── MySQL, MariaDB, PostgreSQL: up to 5 read replicas
└── Oracle, SQL Server: limited read replica support

Cross-region read replicas:
├── Replica in different region (reads closer to users)
├── Useful for global apps and DR
├── Data transferred at standard inter-region rates
└── Can be promoted to standalone DB in disaster

Read Replica promotion:
├── Promote replica to standalone instance (breaks replication)
├── Use for: DR, creating new production instances
└── Promoted replica has its own endpoints
Never promote a read replica as an incident response action on a running
primary. Promotion is a one-way DR operation — use it only when the
primary is completely unavailable. Promoting during an active incident
creates split-brain risk and permanently destroys the replication
relationship.

⚠ Replication lag is ASYNCHRONOUS:
   Write to primary → data appears on replica after lag
   Typically < 1 second, but can grow under heavy write load
   Read-your-own-writes problem: user writes, then reads from
   replica — may not see the write yet. Handle in app code.

Comparison summary:
┌──────────────────┬──────────────────┬──────────────────┐
│ Feature          │ Multi-AZ         │ Read Replicas    │
├──────────────────┼──────────────────┼──────────────────┤
│ Purpose          │ High availability │ Read scaling     │
│ Replication      │ Synchronous      │ Asynchronous     │
│ Readable?        │ ❌ No            │ ✅ Yes           │
```

```
│ Same endpoint?      │ ✅ Yes (DNS flips) │ ❌ No (own endpoint)│
│ Cross-region?       │ ❌ No              │ ✅ Yes              │
│ Failover role       │ ✅ Auto-promotes   │ ❌ Manual promote   │
│ Write latency       │ +1-5ms             │ No impact           │
└────────────────────┴────────────────────┴────────────────────┘

Seconds_Behind_Master stops updating if replication is completely stopped
— a static metric value doesn't mean zero lag. Always cross-check with
SHOW SLAVE STATUS\G directly

Reporting queries on the replica can block replication — the replica SQL
thread competes with read queries for locks. Heavy analytics on a replica
can paradoxically increase its lag.
```

```aws
# Create read replica in same region
aws rds create-db-instance-read-replica \
  --db-instance-identifier mydb-replica-1 \
  --source-db-instance-identifier mydb \
  --db-instance-class db.r5.xlarge \
  --availability-zone us-east-1b

# Cross-region read replica
aws rds create-db-instance-read-replica \
  --db-instance-identifier mydb-eu-replica \
  --source-db-instance-identifier arn:aws:rds:us-east-1:123:db:mydb \
  --region eu-west-1
```

💬 Short Crisp Interview Answer

"RDS is AWS's managed relational database service supporting MySQL, PostgreSQL, Oracle, SQL Server, and MariaDB. Multi-AZ provides high availability through synchronous replication to a standby in a different AZ — on failure, DNS flips to the standby in 60-120 seconds. The standby does NOT serve reads. Read Replicas use asynchronous replication to create readable copies for read scaling — they have their own endpoints, can span regions, and can be manually promoted. The key distinction: Multi-AZ is for HA (synchronous, same endpoint, not readable), Read Replicas are for performance (asynchronous, separate endpoints, readable)."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Multi-AZ standby = NOT readable	Cannot read from standby — it's for HA only
Failover DNS TTL	Keep connection string DNS TTL low (5 seconds) or failover takes longer

Gotcha	Detail
Read replica replication lag	Async means replicas can be behind. Don't read critical data from replica immediately after write
Multi-AZ write latency	Synchronous replication adds ~1-5ms per write
Maintenance window	Multi-AZ: standby patched first → failover → old primary patched
Replica promotion breaks replication	Once promoted, replica is standalone — cannot re-attach to source

---

6.2 DynamoDB — Tables, Items, Partition Keys, GSI/LSI

🟢 What It Is in Simple Terms

DynamoDB is AWS's fully managed NoSQL key-value and document database. No servers to manage, no schema to enforce, scales to any size automatically, and delivers single-digit millisecond performance at any scale. The tradeoff: you must design your data model around your access patterns upfront.

🔍 Why It Exists / What Problem It Solves

Relational databases scale vertically (bigger machines) and struggle with extreme read/write throughput. DynamoDB is designed to scale horizontally across thousands of physical partitions transparently, serving millions of requests per second with consistent low latency.

⚙️ How It Works Internally

```
DynamoDB Internal Architecture:

Your Table
     │
     ├── Partition 1 (key hash range A-M)  ──→  3 copies across AZs
     ├── Partition 2 (key hash range N-T)  ──→  3 copies across AZs
     └── Partition 3 (key hash range U-Z)  ──→  3 copies across AZs

Routing:
Request with partition key "user123"
→ DynamoDB hashes "user123" → determines partition 2
→ Routes request to correct storage node
→ Returns item in single-digit ms

Data model:
├── Tables: top-level container
├── Items: rows/documents (max 400KB per item)
├── Attributes: columns (schemaless — each item can differ)
```

```
└── Keys: uniquely identify items

DynamoDB is schemaless (except for key attributes):
Only primary key attributes need to be defined at table creation.
All other attributes are flexible per item.
```

🧩 Primary Keys — Critical Design Decision

```
Two types of primary keys:

1. Simple Primary Key (Partition Key only):
   └── Table: Users
       PK: userId (string)
       GET user by userId → instant

2. Composite Primary Key (Partition Key + Sort Key):
   └── Table: Orders
       PK: customerId (partition key)
       SK: orderDate#orderId (sort key)

       All of customer's orders stored in same partition
       → Sorted by sort key
       → Query: all orders for customer123
       → Query: orders BETWEEN 2024-01 and 2024-12
       → Get: specific order by customerId + orderId

Partition Key (PK):
├── Determines which PARTITION data is stored on
├── All items with same PK stored together (in partition)
├── Must be unique per item (if no sort key)
└── Choose for HIGH CARDINALITY (many distinct values)
    Low cardinality PK → hot partition problem

Sort Key (SK):
├── Combined with PK = unique identifier per item
├── Items with same PK sorted by SK within the partition
├── Enables range queries: BETWEEN, begins_with, <, >, =
└── Enables 1-to-many relationships in one table

Single-table design example:

┌──────────────────┬──────────────────┬──────────────────────┐
│ PK               │ SK               │ Data                 │
├──────────────────┼──────────────────┼──────────────────────┤
│ USER#user123     │ PROFILE          │ {name, email, ...}   │
│ USER#user123     │ ORDER#2024-001   │ {items, total, ...}  │
│ USER#user123     │ ORDER#2024-002   │ {items, total, ...}  │
│ PRODUCT#p456     │ DETAILS          │ {name, price, ...}   │
│ PRODUCT#p456     │ REVIEW#r789      │ {rating, text, ...}  │
└──────────────────┴──────────────────┴──────────────────────┘
```

```
All access patterns served from one table without joins.
```

🧩 Indexes — GSI and LSI

```
Problem: DynamoDB is fast if you query by primary key.
         What if you need to query by a different attribute?
         e.g., Table keyed by userId, but you need all orders by status.

Solution: Indexes (alternative access patterns)

LSI (Local Secondary Index):
├── Same partition key as base table
├── DIFFERENT sort key
├── Must be defined at TABLE CREATION (cannot add later!)
├── Shares read/write capacity with base table
├── Strongly consistent reads available
├── Limit: 5 LSIs per table
└── Use when: different sort orders for same partition key

Example:
Table: Orders (PK: customerId, SK: orderDate)
LSI: OrdersByStatus (PK: customerId, SK: status)
→ Query: all PENDING orders for customer123

GSI (Global Secondary Index):
├── DIFFERENT partition key (and optionally different sort key)
├── Can be added ANYTIME (not just at creation!)
├── Has its OWN read/write capacity (separate from base table)
├── Eventually consistent reads only
├── Limit: 20 GSIs per table
└── Use when: query by completely different attribute

Example:
Table: Orders (PK: customerId, SK: orderDate)
GSI: GSI1 (PK: status, SK: orderDate)
→ Query: ALL orders with status=SHIPPED across all customers
→ Global = crosses all partitions of the base table

GSI projection types:
├── KEYS_ONLY: only PK, SK, and index keys projected
├── INCLUDE:   PK, SK, index keys + specified attributes
└── ALL:       all attributes projected (costs more storage)

⚠ Critical: GSI projections
Requesting an attribute NOT projected into GSI
→ DynamoDB fetches from base table (extra cost + latency)
→ Include all frequently accessed attributes in projection

LSI vs GSI comparison:
```

```
┌──────────────────┬──────────────────┬──────────────────┐
│ Feature          │ LSI              │ GSI              │
├──────────────────┼──────────────────┼──────────────────┤
│ Partition key    │ Same as table    │ Different        │
│ When to create   │ At table creation │ Anytime          │
│ Consistency      │ Strong or eventual│ Eventual only    │
│ Capacity         │ Shared with table │ Own capacity     │
│ Max per table    │ 5                │ 20               │
└──────────────────┴──────────────────┴──────────────────┘
```

```python
import boto3

dynamodb = boto3.resource('dynamodb')

# Create table with GSI
dynamodb.create_table(
    TableName='Orders',
    KeySchema=[
        {'AttributeName': 'customerId', 'KeyType': 'HASH'},
        {'AttributeName': 'orderDate',  'KeyType': 'RANGE'}
    ],
    AttributeDefinitions=[
        {'AttributeName': 'customerId', 'AttributeType': 'S'},
        {'AttributeName': 'orderDate',  'AttributeType': 'S'},
        {'AttributeName': 'status',     'AttributeType': 'S'},
    ],
    GlobalSecondaryIndexes=[{
        'IndexName': 'StatusDateIndex',
        'KeySchema': [
            {'AttributeName': 'status',    'KeyType': 'HASH'},
            {'AttributeName': 'orderDate', 'KeyType': 'RANGE'}
        ],
        'Projection': {'ProjectionType': 'ALL'},
        'BillingMode': 'PAY_PER_REQUEST'
    }],
    BillingMode='PAY_PER_REQUEST'
)

# Query GSI: get all SHIPPED orders in date range
table = dynamodb.Table('Orders')
response = table.query(
    IndexName='StatusDateIndex',
    KeyConditionExpression='#s = :status AND orderDate BETWEEN :start AND
:end',
    ExpressionAttributeNames={'#s': 'status'},
    ExpressionAttributeValues={
        ':status': 'SHIPPED',
        ':start':  '2024-01-01',
        ':end':    '2024-12-31'
    }
)
```

💬 Short Crisp Interview Answer

"DynamoDB is AWS's fully managed NoSQL database delivering single-digit millisecond latency at any scale. The data model uses tables with items — each item uniquely identified by its primary key. Simple primary keys use only a partition key; composite keys add a sort key enabling range queries and one-to-many relationships within a partition. The partition key determines physical data placement — high cardinality keys prevent hot partitions. LSIs keep the same partition key with a different sort key, must be created at table creation, and support strong consistency. GSIs use a completely different partition key, can be added anytime, are eventually consistent only, and have their own read/write capacity. The fundamental rule: design your primary key and indexes around your access patterns first."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
LSI cannot be added later	Must define at table creation — cannot add LSI to existing table
GSI eventually consistent	Cannot do strongly consistent reads on a GSI
Item size limit	400KB per item — large documents should be in S3 with reference in DynamoDB
GSI capacity separate	GSI throttled → base table writes that update GSI also throttled
Low cardinality PK	All data on one partition → hot partition → throttling
PK range queries	Only sort key supports range operators. PK supports equality only

---

6.3 RDS — Parameter Groups, Option Groups, Automated Backups, Snapshots

---

🟢 What It Is in Simple Terms

The operational knobs for tuning and protecting RDS databases. Parameter groups customize database engine settings. Option groups add extra database features. Backups and snapshots give you point-in-time recovery and cross-account protection.

🧩 Parameter Groups

```
Parameter Groups = database engine configuration settings
```

```
Every RDS instance is associated with one parameter group.
A parameter group contains database engine parameters:
├── max_connections (MySQL)
├── shared_buffers (PostgreSQL)
├── innodb_buffer_pool_size (MySQL)
├── log_min_duration_statement (PostgreSQL — slow query logging)
└── 100s of other engine-specific settings

Two types of parameters:
├── Static parameters:  require DB restart to take effect
└── Dynamic parameters: apply immediately (no restart needed)
```

```
# Create custom parameter group (default groups cannot be modified)
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-postgres-params \
  --db-parameter-group-family postgres15 \
  --description "Production PostgreSQL 15 parameters"

# Modify parameters
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-postgres-params \
  --parameters \

ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=i
mmediate \

ParameterName=shared_preload_libraries,ParameterValue=pg_stat_statements,A
pplyMethod=pending-reboot \
    ParameterName=max_connections,ParameterValue=500,ApplyMethod=pending-
reboot

# Associate parameter group with instance
aws rds modify-db-instance \
  --db-instance-identifier mydb \
  --db-parameter-group-name prod-postgres-params \
  --apply-immediately
```

🧩 Option Groups

```
Option Groups = additional database features/plugins
Primarily used for Oracle and SQL Server (licensed features).
Also used for MySQL plugins.

Examples:
├── Oracle:     OEM (Oracle Enterprise Manager), TDE encryption
├── SQL Server: SQL Server Audit, TDE
├── MySQL:      MariaDB Audit Plugin, Memcached integration
└── PostgreSQL: uses parameter groups for extensions instead
```

```
aws rds create-option-group \
  --option-group-name prod-oracle-options \
  --engine-name oracle-ee \
  --major-engine-version 19 \
  --option-group-description "Oracle EE options"

aws rds add-option-to-option-group \
  --option-group-name prod-oracle-options \
  --options OptionName=OEM
```

🧩 Automated Backups

```
Automated Backups:
├── Enabled by default on all RDS instances
├── Daily full backup during backup window (low-traffic period)
├── Continuous transaction log backups (every 5 minutes)
├── Retention: 1-35 days (default 7 days)
├── Stored in S3 (managed by AWS — you don't see the bucket)
├── Free storage up to size of DB
└── Enables Point-in-Time Recovery (PITR)

Point-in-Time Recovery (PITR):
├── Restore to ANY second within retention period
├── Restores to a NEW DB instance (not in-place!)
├── Combines: last full backup + transaction logs
└── Most recent restorable time: typically 5 minutes ago

⚠ PITR creates a NEW instance — not in-place restore.
   After recovery, update connection strings in your app.
   Old instance still running (you pay for both until you delete old).

Backup window:
├── 30-minute window when daily backup runs
├── During this window: brief I/O suspension may occur
│   (seconds, not minutes — on single-AZ instances)
├── Multi-AZ: backup taken from standby (no I/O impact on primary)
└── Set to low-traffic time: e.g., 02:00-02:30 UTC
```

```
# Restore to point in time (creates new instance)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier mydb \
  --target-db-instance-identifier mydb-recovered \
  --restore-time 2024-01-15T12:00:00Z
```

🧩 DB Snapshots

```
DB Snapshots = manual point-in-time backups (user-initiated)

Snapshot properties:
├── Full copy of DB storage at that moment
├── RETAINED INDEFINITELY until manually deleted
│   (unlike automated backups which expire)
├── Can be copied cross-region
├── Can be shared with other AWS accounts
├── Can be encrypted with different KMS key when copying
└── Stored in S3 (AWS managed)

⚠ Single-AZ snapshot: brief I/O suspension during snapshot.
   Multi-AZ snapshot: taken from standby — no I/O impact on primary.

Automated vs Manual snapshots:
┌──────────────────┬──────────────────┬──────────────────────┐
│ Feature          │ Automated Backup │ Manual Snapshot      │
├──────────────────┼──────────────────┼──────────────────────┤
│ Created by       │ AWS automatically│ You manually         │
│ Retention        │ 1-35 days        │ Forever (until delete)│
│ PITR             │ ✅ Yes           │ ❌ No (full only)    │
│ Cross-region     │ ❌ Not directly  │ ✅ Yes (copy first)  │
│ Cross-account    │ ❌ Not directly  │ ✅ Yes               │
│ On DB deletion   │ Optionally kept  │ Kept until deleted   │
└──────────────────┴──────────────────┴──────────────────────┘
```

```aws
# Take snapshot before major change
aws rds create-db-snapshot \
  --db-instance-identifier mydb \
  --db-snapshot-identifier mydb-before-schema-migration

# Copy snapshot cross-region (for DR)
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier arn:aws:rds:us-east-1:123:snapshot:mydb-
snap \
  --target-db-snapshot-identifier mydb-snap-dr \
  --source-region us-east-1 \
  --region us-west-2 \
  --kms-key-id arn:aws:kms:us-west-2:123:key/...

# Share snapshot with another AWS account
aws rds modify-db-snapshot-attribute \
  --db-snapshot-identifier mydb-snap \
  --attribute-name restore \
  --values-to-add 999888777666

# Restore from snapshot
aws rds restore-db-instance-from-db-snapshot \
```

```
  --db-instance-identifier mydb-restored \
  --db-snapshot-identifier mydb-before-schema-migration
```

🧩 RDS Monitoring

```
RDS Monitoring = tracking performance, health, and resource usage of your
database

📊 Core Monitoring Services:
├── Amazon CloudWatch (default monitoring)
├── Enhanced Monitoring (OS-level metrics)
├── Performance Insights (query-level analysis)
└── RDS Events (notifications for state changes)

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

📈 Amazon CloudWatch Metrics
(Default, basic monitoring)

Key metrics:
├── CPUUtilization → CPU usage of DB instance
├── FreeStorageSpace → available disk space
├── DatabaseConnections → active connections
├── ReadIOPS / WriteIOPS → disk I/O operations
├── ReadLatency / WriteLatency → I/O delays
├── DiskQueueDepth → number of pending I/O requests waiting to be
processed
├── NetworkReceiveThroughput / Transmit → network usage
└── FreeableMemory → available RAM

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

🧠 DiskQueueDepth (Important)

Definition:
├── Number of read/write operations waiting in queue for disk access

Interpretation:
├── Low (0–1) → Healthy
├── Moderate (2–10) → Normal under load
└── High (>10 consistently) → Disk I/O bottleneck

Causes of High Queue Depth:
├── Insufficient IOPS (gp2/gp3 limits)
├── High read/write workload
├── Slow storage type
└── Inefficient queries (full table scans)

Solutions:
├── Increase IOPS (gp3 / Provisioned IOPS)
```

```
├── Optimize queries (indexes, tuning)
├── Scale instance (better throughput)
└── Use caching (e.g., ElastiCache)

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

⚙️ Enhanced Monitoring
(OS-level visibility inside DB instance)

Features:
├── Granular metrics (1s to 60s)
├── CPU breakdown (user/system/idle)
├── Memory usage (actual OS-level)
├── Disk I/O stats per process
└── Uses CloudWatch Logs agent

Use case:
└── Debugging CPU, memory, and process-level issues

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

🔥 Performance Insights
(Query-level monitoring)

Features:
├── Identifies top SQL queries by load
├── DB load visualization (DBLoad)
├── Wait events (CPU, IO, locks)
├── Historical performance analysis
└── Detects slow queries & bottlenecks

Use case:
└── Query optimization and tuning

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

🔔 RDS Events
(Notification system for DB changes)

Examples:
├── DB instance restart
├── Multi-AZ failover
├── Backup completion
├── Maintenance activities
└── Storage full warning

Integration:
└── SNS → Email / SMS / Lambda

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

🆚 Monitoring Types Comparison

┌──────────────────────┬──────────────────────┬──────────────────────────┐
```

```
│ Feature              │ CloudWatch           │ Enhanced Monitoring     │
├──────────────────────┼──────────────────────┼─────────────────────────┤
│ Level                │ DB instance          │ OS-level                │
│ Granularity          │ 1 min (default)      │ 1 sec                   │
│ Metrics source       │ Hypervisor           │ Inside OS               │
│ Use case             │ General monitoring   │ Deep troubleshooting    │
└──────────────────────┴──────────────────────┴─────────────────────────┘

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

🧠 Key Interview Points:
├── CloudWatch = default monitoring
├── Enhanced Monitoring = OS-level visibility
├── Performance Insights = query-level analysis
├── DiskQueueDepth helps identify I/O bottlenecks
├── Multi-AZ failover tracked via RDS Events
└── Combine all tools for full observability

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

⚠️ Best Practices:
├── Enable Enhanced Monitoring in production
├── Use Performance Insights for slow queries
├── Set CloudWatch Alarms:
│   ├── CPUUtilization > 80%
│   ├── FreeStorageSpace low
│   ├── DatabaseConnections high
│   └── DiskQueueDepth high
├── Monitor during peak load
└── Enable DB logs (slow query, error logs)

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

🎯 Troubleshooting Scenarios:

1. CPU high
   └── Check: queries (Performance Insights), connections

2. DB slow but CPU low
   └── Check: DiskQueueDepth, Read/Write Latency, IOPS

3. Connections high
   └── Check: connection pooling, max_connections

4. Storage full
   └── Enable autoscaling or increase storage

5. High latency
   └── Check: IOPS limits, queue depth, slow queries
```

💬 Short Crisp Interview Answer

"Parameter groups configure database engine settings — things like max_connections, buffer sizes, and slow query logging. Static parameters require a restart; dynamic apply immediately. Option groups add licensed features, primarily for Oracle and SQL Server. Automated backups run daily in a backup window with continuous transaction log backups, enabling Point-in-Time Recovery to any second within the retention period (1-35 days). PITR always restores to a new instance. Manual snapshots are user-initiated, retained indefinitely until deleted, can be copied cross-region and shared cross-account. Key difference: automated backups enable PITR and expire automatically; manual snapshots are full copies kept forever — take one before any major schema change."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Default parameter group immutable	Can't modify default. Create a custom group first
Parameter group change requires restart	Static parameters only apply after DB reboot
PITR = new instance	Restores to new DB, not in-place. Update connection strings
Snapshot on deletion	When deleting RDS, you're prompted for a final snapshot. Don't skip it
Single-AZ backup I/O suspension	Brief I/O pause during backup. Use Multi-AZ for production
Automated backup retention 0	Setting to 0 disables backups AND deletes all existing automated backups

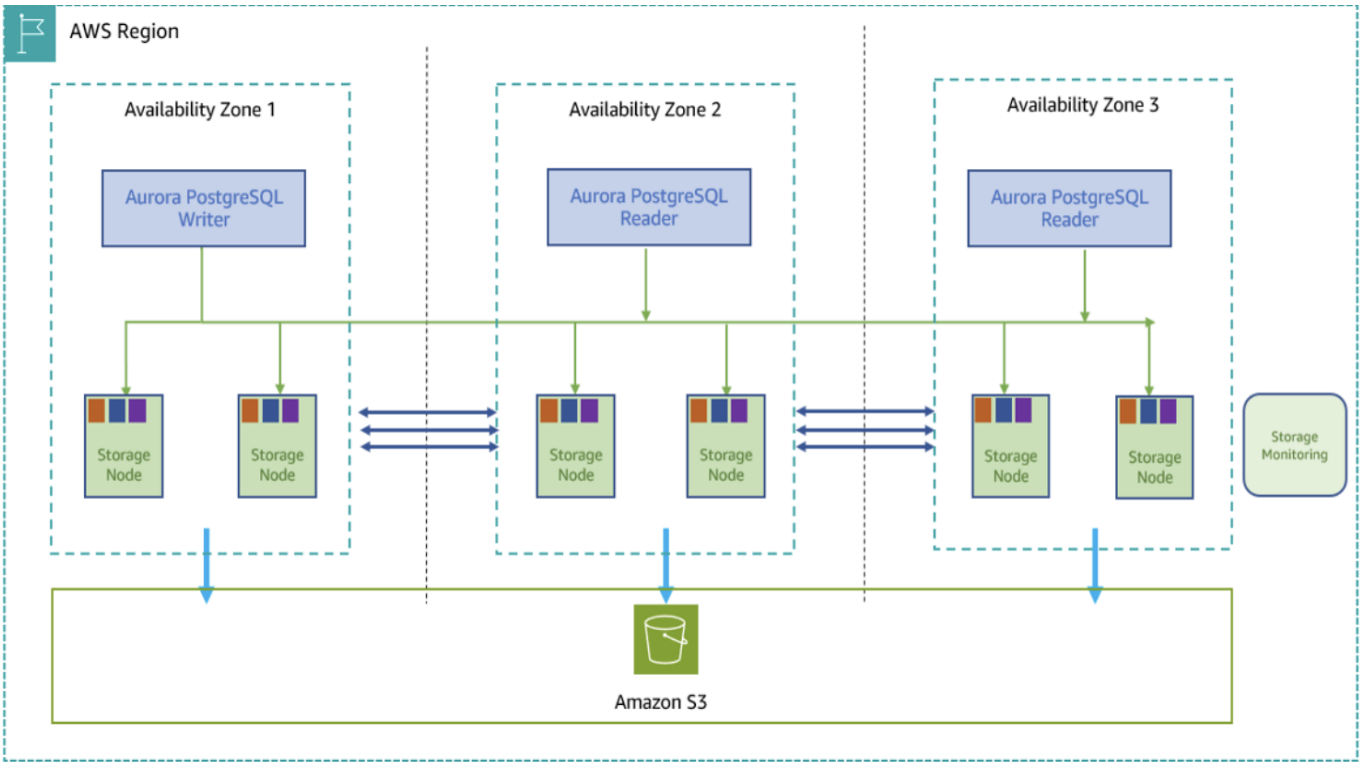
---

6.4 Aurora — Architecture, Aurora Serverless, Global Database

---

🟢 What It Is in Simple Terms

Aurora is AWS's cloud-native relational database — MySQL and PostgreSQL compatible but rebuilt from the ground up for the cloud. It's not MySQL running on AWS — it's a new storage engine with a shared distributed storage layer that gives it fundamentally better durability, performance, and scaling characteristics than standard RDS.



🔍 Why It Exists / What Problem It Solves

Standard RDS replication copies the entire database storage between primary and replica. Aurora separates compute from storage — all instances share the same distributed storage layer. This means faster failover, better durability, and replicas that are instantly in sync without data copying.

⚙️ Aurora Architecture

```
Aurora Storage Architecture:

Compute Layer (you manage):

┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  Writer      │    │  Reader 1    │    │  Reader 2    │
│  (Primary)   │    │  (Replica)   │    │  (Replica)   │
└──────────────┘    └──────────────┘    └──────────────┘
       │                   │                   │
       └───────────────────┼───────────────────┘
                           │
                           ▼
Storage Layer (AWS fully manages — you never see this):

┌──────────────────────────────────────────────────────┐
│           Aurora Distributed Storage                 │
│                                                      │
│  AZ-a:          AZ-b:          AZ-c:                 │
│  [copy1][copy2] [copy3][copy4] [copy5][copy6]        │
│                                                      │
│  6 copies of data across 3 AZs                       │
│  Quorum: 4 of 6 for writes, 3 of 6 for reads         │
│  Can tolerate: 2 copy failures for writes            │
```

```
│                3 copy failures for reads                  │
│  Auto-heals corrupted copies automatically               │
└──────────────────────────────────────────────────────────┘

Aurora storage properties:
├── Auto-grows in 10GB increments (no pre-provisioning)
├── Max: 128TB
├── You pay only for what you use
└── Storage I/O costs extra (charged per million I/O requests)
    ⚠ Different from RDS which bundles I/O in instance price.
    For I/O-heavy workloads, standard RDS may be cheaper.

Why this architecture is better than standard RDS:
├── No replication of storage between primary and replica
│   (all share same storage — replication is just log pointers)
├── Reader replicas up-to-date in milliseconds (not seconds)
├── Failover in ~30 seconds (no data to copy)
└── Durability: 6 copies, survives 2 AZ failures
```

🧩 Aurora Cluster Components

```
Aurora Cluster:
├── Cluster Endpoint:    always points to current writer
├── Reader Endpoint:     load-balanced across all readers
├── Instance Endpoints:  direct to specific instance (avoid usually)
└── Custom Endpoints:    your-defined subset of instances

Aurora Replicas:
├── Up to 15 read replicas (vs 5 for RDS)
├── Sub-10ms replication lag (vs seconds for RDS)
├── Automatic failover: replica promoted to writer if primary fails
└── Failover priority: assign tiers 0-15 per replica
    (tier 0 = first to be promoted, tier 15 = last)
```

```
# Auto-scaling Aurora read replicas
aws application-autoscaling register-scalable-target \
  --service-namespace rds \
  --resource-id cluster:my-aurora-cluster \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --min-capacity 1 \
  --max-capacity 15

aws application-autoscaling put-scaling-policy \
  --policy-name aurora-replica-scaling \
  --service-namespace rds \
  --resource-id cluster:my-aurora-cluster \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --policy-type TargetTrackingScaling \
```

```
  --target-tracking-scaling-policy-configuration '{
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "RDSReaderAverageCPUUtilization"
    },
    "TargetValue": 60.0,
    "ScaleInCooldown": 300,
    "ScaleOutCooldown": 300
  }'
```

🧩 Aurora Serverless v2

```
Aurora Serverless v2:
├── Compute scales automatically based on actual load
├── Scale range: 0.5 ACU (Aurora Capacity Units) to 128 ACUs
│   1 ACU ≈ 2 GB RAM
├── Scales in fine-grained increments (~seconds)
├── Works in the same cluster as provisioned instances
│   (can mix serverless and provisioned!)
└── Minimum 0.5 ACU — does NOT scale to zero (always a cost)

Cost:
├── ~$0.12/ACU-hour (us-east-1 approximately)
└── Scales quickly → cost tracks actual usage

Aurora Serverless v1 (legacy):
├── Scales to ZERO (no cost when idle)
├── But cold start from zero = 15-30 seconds to resume
├── Cannot have read replicas
└── Use v2 for all new workloads

When to use Serverless v2:
├── Dev/staging environments with variable load
├── New SaaS applications (uncertain traffic patterns)
├── Reporting databases (queried occasionally)
└── When you don't want to right-size instance type

Provisioned vs Serverless v2:
├── Provisioned: fixed instance, predictable cost,
│              better for stable high-load production
└── Serverless v2: variable cost, flexible, good for variable load
```

🧩 Aurora Global Database

```
Aurora Global Database:
├── ONE Aurora cluster spans MULTIPLE AWS regions
├── Primary region: handles all writes
├── Secondary regions (up to 5): read-only, replicated async
```

```
  ├── Replication lag: < 1 second (typically ~100ms) across regions
  ├── RPO (Recovery Point Objective): ~1 second on regional failure
  └── RTO (Recovery Time Objective): < 1 minute on regional failure

  Architecture:
  Region: us-east-1 (primary)          Region: eu-west-1 (secondary)
  ┌──────────────────────────┐          ┌──────────────────────────┐
  │ Aurora Writer            │          │ Aurora Reader(s)         │
  │ Aurora Readers           │──────────► (read-only replication)  │
  │ Aurora Storage (6 copies)│          │ Aurora Storage (6 copies)│
  └──────────────────────────┘          └──────────────────────────┘
         ↕ < 1 second replication lag

  Use cases:
  ├── Global apps: US users read from US, EU reads from EU
  ├── Disaster recovery: promote secondary if primary region fails
  └── Compliance: keep data copies in specific regions

  Failover procedure (manual):
  1. Detect primary region failure
  2. In secondary region, remove from global DB (detach)
  3. Promote secondary to standalone read-write cluster
  4. Update application connection strings to secondary endpoint
  5. Re-attach recovered primary as secondary when it recovers

  ⚠ Failover is NOT automatic — no auto-promotion yet.
     Must manually trigger. Aim for < 1 minute manual process.
```

💬 Short Crisp Interview Answer

"Aurora is a cloud-native relational database compatible with MySQL and PostgreSQL. Its key architectural innovation is separating compute from storage — all instances share a single distributed storage layer with 6 copies across 3 AZs. This gives sub-10ms replication lag, 30-second failover (no data to copy), and 128TB auto-scaling storage. Aurora Serverless v2 automatically scales compute in fine increments based on actual load — good for variable workloads, though it doesn't scale to zero. Aurora Global Database replicates across up to 5 regions with under 1 second lag, giving near-zero RPO for global DR and read scaling for globally distributed users. Failover is currently manual."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Aurora I/O costs	I/O billed separately per million requests. High-I/O workloads may be cheaper on standard RDS
Serverless v2 doesn't scale to zero	Minimum 0.5 ACU always running = ~$43/month minimum

Gotcha	Detail
Global DB failover is manual	No automatic promotion — must manually detach and promote secondary
Reader endpoint load-balanced	Not session-pinned. Connections may hit different replicas across reconnects
Custom endpoints	Use for routing analytics workloads to specific replicas without affecting OLTP

- **Buffer pool cold start is mandatory planning for any Aurora blue/green deployment** — always pre-warm the green cluster with representative queries before cutting over traffic. Skipping this step causes latency doubling that looks like under-provisioning.
- `BufferCacheHitRatio` **is your leading indicator during cutover** — set a CloudWatch alarm below 90% and treat it as a rollback trigger. Don't wait for user complaints to detect a cold buffer pool.
- **Aurora Global Database failover is always manual** — there is no automatic promotion — the secondary cluster remains read-only indefinitely during a primary region outage. This is by design to prevent split-brain. Always have a pre-built runbook and SSM Automation document ready before you need it.
- **Writer endpoint changes after promotion** — applications must be updated to point to the new endpoint. Always use Route 53 CNAME abstraction or Parameter Store for DB endpoints — never hardcode Aurora endpoints in application configs.
- **Pre-set DNS TTL to 30 seconds before any planned failover** — if TTL is 300 seconds (default), DNS propagation alone adds 5 minutes to your RTO. Change TTL at least 24 hours before any expected failover event.

6.5 DynamoDB — Capacity Modes, DAX, Streams, TTL

🟢 What It Is in Simple Terms

DynamoDB's operational controls: how you pay for throughput (provisioned vs on-demand), how to cache results for microsecond reads (DAX), how to react to data changes in real time (Streams), and how to auto-expire old data (TTL).

🧩 Capacity Modes

```
DynamoDB measures throughput in:
├── RCU (Read Capacity Unit):
│   1 RCU = 1 strongly consistent read/sec for item ≤ 4KB
│         = 2 eventually consistent reads/sec for item ≤ 4KB
│   Reading 8KB item strongly consistent:  2 RCUs
│   Reading 8KB item eventually consistent: 1 RCU
│
└── WCU (Write Capacity Unit):
```

```
    1 WCU = 1 write/sec for item ≤ 1KB
    Writing 3.5KB item: 4 WCUs (round up to next 1KB)
    Writing 0.5KB item: 1 WCU

Mode 1: Provisioned Capacity
├── Specify RCUs and WCUs in advance
├── Pay per provisioned unit per hour (whether used or not)
├── Auto-scaling adjusts based on utilization target
├── Burst capacity: absorb short spikes using saved capacity
│   (capacity saved when under-utilized, up to 5 minutes)
└── Best for: predictable, stable workloads

Mode 2: On-Demand Capacity
├── No capacity planning required
├── Scales instantly to any load
├── Pay per request: reads $0.25/million, writes $1.25/million
├── ~5x more expensive at high sustained throughput
└── Best for: unpredictable workloads, new tables, spiky traffic

Cost comparison (1M reads + 1M writes per day):

Provisioned (auto-scaling):
RCU needed: ~12 sustained
Cost: ~$0.23/day

On-Demand:
1M reads × $0.25/M + 1M writes × $1.25/M = $1.50/day
(~5x more expensive for stable load)

⚠ Auto-scaling is REACTIVE (responds within 60-90 seconds).
   Bursts faster than scale-out can exhaust capacity.
   Use On-Demand for very spiky, unpredictable patterns.
```

🧩 DAX — DynamoDB Accelerator

```
DAX = in-memory caching layer for DynamoDB
     Purpose: microsecond response for read-heavy workloads
     (vs DynamoDB's single-digit millisecond)

How DAX works:
Cache HIT:  App → DAX → returns cached item (microseconds)
Cache MISS: App → DAX → DAX fetches from DynamoDB → caches → returns

DAX architecture:
├── Cluster of nodes (1 primary + 1-9 read replicas)
├── Deployed in your VPC (private subnet)
├── Uses DAX SDK (drop-in replacement for DynamoDB SDK)
├── Item cache: individual GetItem results
├── Query cache: Query/Scan results
└── Default TTL: 5 minutes (item cache), 1 minute (query cache)
```

```
What DAX is good for:
├── Read-heavy workloads (product catalogs, leaderboards)
├── Latency-sensitive reads (gaming, financial tickers)
└── Repeated reads of same items (popular content)

What DAX is NOT good for:
├── Write-heavy workloads (DAX doesn't help writes)
├── Strongly consistent reads (bypasses cache → goes direct to DDB)
└── Rarely accessed items (cache misses = no benefit)

⚠ DAX returns cached data on GetItem.
   If strongly consistent reads are required, DAX bypasses cache.
   Strongly consistent = goes directly to DynamoDB (no DAX benefit).

DAX vs ElastiCache:
├── DAX: DynamoDB-specific, drop-in SDK, simpler, write-through
└── ElastiCache: generic, more control, complex caching logic,
                 Redis data structures
```

🧩 DynamoDB Streams

```
DynamoDB Streams = time-ordered log of item changes

What it captures:
├── INSERT: new item image
├── MODIFY: before + after images
└── DELETE: before image (what was deleted)

Stream retention: 24 hours

Stream record content (StreamViewType):
├── KEYS_ONLY:         only primary key attributes
├── NEW_IMAGE:         entire item after modification
├── OLD_IMAGE:         entire item before modification
└── NEW_AND_OLD_IMAGES: both (most useful for change tracking)
```

```
# Enable streams on table
aws dynamodb update-table \
  --table-name Orders \
  --stream-specification
StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES
```

```
Processing streams:
├── Lambda Event Source Mapping → Lambda consumes stream
├── One Lambda invocation per shard
```

```
└── Shards: roughly 1 per 1000 WCU (or 1 per table if small)

Common stream use cases:
├── Replication:    sync to another table or region
├── Event-driven:  trigger notifications on item changes
│                  (new order → send confirmation email)
├── Aggregation:   maintain counts/totals in separate table
├── Audit trail:   log all changes for compliance
└── Search:        propagate changes to OpenSearch/Elasticsearch

Streams + Lambda example:
Orders table → Stream → Lambda
  → SES (send order confirmation)
  → SNS (notify fulfillment)
  → OpenSearch (index for search)

⚠ Streams ordered within a shard, not across shards.
   For strict global ordering, use Kinesis Data Streams for DynamoDB.

⚠ TTL deletions appear in streams:
   Filter by: record.userIdentity.principalId == "dynamodb.amazonaws.com"
```

🧩 TTL — Time to Live

```
TTL = automatically delete items after a Unix timestamp

How it works:
├── Add Unix timestamp attribute to items
├── Enable TTL on that attribute name for the table
├── DynamoDB background process deletes expired items
└── Deletion happens within 48 hours of expiry timestamp
    (not instantaneous — background process)
```

```
# Enable TTL on table
aws dynamodb update-time-to-live \
  --table-name Sessions \
  --time-to-live-specification Enabled=true,AttributeName=expiresAt
```

```
import time

# Add TTL attribute to item
table.put_item(Item={
    'sessionId': 'abc123',
    'userId':    'user456',
    'data':      {...},
```

```
    'expiresAt': int(time.time()) + 3600  # expires in 1 hour
})
```

```
TTL use cases:
├── Session tokens (expire after 24 hours)
├── Temporary data (job results, cache entries)
├── GDPR compliance (delete personal data after retention period)
└── Preventing table growth for time-series data

⚠ TTL is EVENTUALLY applied (within 48 hours — not exact).
   Expired-but-not-yet-deleted items still returned by Query/Scan.
   Always filter expired items in your application code.
```

💬 Short Crisp Interview Answer

"DynamoDB has two capacity modes. Provisioned requires you to specify RCUs and WCUs upfront and is cheaper for stable workloads. On-Demand requires no capacity planning, scales instantly, but costs 5x more per request at sustained load. DAX is DynamoDB's in-memory caching layer — drop-in SDK replacement that delivers microsecond reads, but doesn't help writes and bypasses cache for strongly consistent reads. DynamoDB Streams captures a 24-hour ordered log of all item changes that you process with Lambda for event-driven patterns like search indexing or order notifications. TTL auto-expires items by a Unix timestamp attribute — deletions happen within 48 hours and expired items may still be returned by queries, so filter in application code."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Auto-scaling is reactive	Can't absorb instantaneous spikes — scale-out takes 60-90 seconds
On-Demand more expensive at scale	5x cost at high sustained load. Switch to provisioned when stable
DAX + strong consistency	Strongly consistent reads bypass DAX entirely
TTL is approximate	Deletions happen within 48 hours. Don't rely on exact timing
TTL items still returned	Expired-but-not-deleted items show in Scan/Query. Filter in code
Streams 24-hour window	Must process within 24 hours or records are permanently lost

---

6.6 ElastiCache — Redis vs Memcached, Cluster Mode

🟢 What It Is in Simple Terms

ElastiCache is AWS's managed in-memory caching service. You get either Redis or Memcached fully managed — no server provisioning, built-in replication, failover, and monitoring. It sits in front of your database and serves frequent reads at microsecond speed.

🔍 Why It Exists / What Problem It Solves

Databases are fast for persistent queries but slow compared to in-memory access. For frequently accessed data (user sessions, product catalog, leaderboards), going to the database every time wastes time and money. ElastiCache caches results in memory and returns them in sub-millisecond time.

⚙️ Common Caching Patterns

```
Lazy Loading (Cache-Aside):
App requests data
  → Check cache
    HIT:  return from cache (fast)
    MISS: query database → store in cache → return
✅ Only caches data that's actually requested
⚠ First request always slow (cache miss)
⚠ Stale data if database updated without cache invalidation

Write-Through:
On every database write → also write to cache
✅ Cache always up to date
⚠ Every write takes longer (write to both DB and cache)
⚠ Caches data that may never be read (wastes memory)

Write-Behind (Write-Back):
Write to cache first → asynchronously write to database
✅ Fastest write latency
⚠ Risk of data loss if cache fails before async write

TTL (Time-to-Live):
Set expiry on cached items → forces re-fetch from DB after expiry
→ Balances freshness vs performance
```

🧩 Redis vs Memcached

```
┌──────────────────────────┬──────────────────────┬──────────────────────────
┐
│ Feature                  │ Redis                │ Memcached
│
├──────────────────────────┼──────────────────────┼──────────────────────────
┤
```

```
│ Data structures          │ String, Hash, List, │ String only
│                          │ Set, Sorted Set,    │
│                          │ Stream, HyperLogLog │
│ Persistence              │ ✅ RDB + AOF        │ ❌ Memory only
│ Replication              │ ✅ Primary + replica │ ❌ No replication
│ Multi-AZ failover        │ ✅ Automatic        │ ❌ No
│ Pub/Sub messaging        │ ✅ Yes              │ ❌ No
│ Lua scripting            │ ✅ Yes              │ ❌ No
│ Transactions (MULTI)     │ ✅ Yes              │ ❌ No
│ Geospatial               │ ✅ Yes              │ ❌ No
│ Cluster mode             │ ✅ Yes              │ ✅ Yes (sharding)
│ Multi-threaded           │ ❌ Single-threaded  │ ✅ Multi-threaded
└──────────────────────────┴─────────────────────┴──────────────────────
┘

Choose Redis when:
├── You need persistence (survive restart)
├── You need HA with automatic Multi-AZ failover
├── You need rich data structures (sorted sets for leaderboards)
├── You need pub/sub messaging
└── 99% of real production use cases → Redis

Choose Memcached when:
├── Pure, simple object caching only
├── Multi-threaded performance for ultra-high throughput
└── No persistence needed

In practice: use Redis for almost everything.
```

🧩 Redis Key Features

```
Redis data structures and use cases:

String:      session tokens, simple key-value cache
Hash:        user profile (userId → {name, email, role})
List:        message queues, activity feeds, chat history
Set:         unique visitors, tags, social graph followers
Sorted Set:  leaderboards, priority queues
```

```
              ZADD leaderboard 9800 "player123"
              ZRANGE leaderboard 0 9 WITHSCORES REV  # top 10
Stream:       event sourcing, activity logs
HyperLogLog:  approximate unique count (memory efficient)
Geospatial:   location-based queries (stores within 5km)

Redis persistence:
├── RDB (Redis Database): periodic point-in-time snapshots
│   Compact, fast recovery. May lose data between snapshots.
└── AOF (Append-Only File): log every write operation
    More durable but larger file and slower recovery.
    Use both: RDB for backups, AOF for durability.

Redis replication:
├── 1 primary + up to 5 read replicas
├── Replication: asynchronous
├── Failover: automatic (replica promoted) in ~30 seconds
└── Multi-AZ: primary in one AZ, replica in another
```

🧩 Cluster Mode

```
Redis Cluster Mode:
├── Shards data across multiple primary nodes (horizontal scaling)
├── Each shard: 1 primary + 0-5 replicas
├── Keyspace divided into 16,384 hash slots
├── Each shard handles a subset of hash slots
└── Up to 500 nodes total

Non-clustered Redis (Replication Group):
├── 1 primary node
├── Up to 5 read replicas
├── All data on one shard
└── Scales vertically (bigger instance type)

Cluster Mode ON vs OFF:
┌──────────────────┬──────────────────┬──────────────────┐
│ Feature          │ Cluster Mode OFF │ Cluster Mode ON  │
├──────────────────┼──────────────────┼──────────────────┤
│ Shards           │ 1                │ 1-500            │
│ Max memory       │ 1 node's RAM     │ Sum of all shards │
│ Write scaling    │ ❌ Vertical only  │ ✅ Horizontal sharding │
│ Multi-key commands │ ✅ Any key      │ ⚠ Same slot only │
│ MULTI/EXEC across │ ✅ Any key      │ ❌ Same slot only │
└──────────────────┴──────────────────┴──────────────────┘

⚠ Cluster Mode + multi-key operations:
   MGET, MSET, pipelines crossing multiple shards = NOT supported.
   Use hash tags: {user123}.session and {user123}.profile
   forces both keys to the same hash slot.
```

```
Use Cluster Mode when:
├── Dataset exceeds single node RAM
└── Write throughput exceeds single primary capacity
```

```aws
# Create Redis cluster with Multi-AZ and replication
aws elasticache create-replication-group \
  --replication-group-id prod-redis \
  --description "Production Redis cluster" \
  --engine redis \
  --cache-node-type cache.r6g.xlarge \
  --num-cache-clusters 3 \
  --multi-az-enabled \
  --automatic-failover-enabled \
  --cache-subnet-group-name prod-redis-subnet \
  --security-group-ids sg-redis \
  --at-rest-encryption-enabled \
  --transit-encryption-enabled \
  --auth-token "SuperSecretRedisPassword123!"
```

💬 Short Crisp Interview Answer

"ElastiCache is AWS's managed in-memory caching service offering Redis and Memcached. In almost all production cases, Redis is the right choice — rich data structures (sorted sets for leaderboards, hashes for user profiles), persistence (RDB snapshots, AOF logging), replication with automatic Multi-AZ failover, and pub/sub. Memcached is simpler and multi-threaded but has no persistence or replication. For Redis scaling: non-clustered mode has one primary with up to 5 read replicas for vertical scaling. Cluster Mode shards data across up to 500 nodes enabling horizontal write scaling when data exceeds single-node RAM. Cluster mode gotcha: multi-key commands across different hash slots fail — use hash tags to force related keys to the same slot."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Memcached no persistence	Node restart = data loss
Redis async replication	Small window of potential data loss on failover
Cluster mode multi-key ops	MGET/MSET/transactions crossing shards fail. Use hash tags
ElastiCache in VPC only	Cannot be accessed from public internet directly
In-transit encryption	Must enable at creation — cannot add to existing cluster
Eviction policy	Set allkeys-lru for full caches to prevent errors
Redis AUTH	Enable --auth-token or --transit-encryption-enabled for security

- **CROSSSLOT error only appears in Cluster Mode** — single-node Redis and non-clustered replication groups don't have this constraint. Always test multi-key commands against a cluster-mode Redis instance in development, not a single-node instance.
- **Hash tags can create hot shards if overused** — if all keys share the same hash tag, they all land on one shard, defeating the horizontal scaling purpose of Cluster Mode. Use hash tags only for keys that require co-location for specific multi-key operations.
- **Pipelining individual GETs is a valid temporary workaround** — but hash tags are the correct architectural fix. Document the temporary workaround with a ticket to implement hash tags properly.

6.7 Aurora — Storage Architecture, Fast Failover, Parallel Query

---

🟢 What It Is in Simple Terms

A deeper look at what makes Aurora's storage fundamentally different, why it fails over so much faster than RDS, and how Parallel Query lets it run analytics without a separate data warehouse.

🧩 Storage Architecture Deep Dive

```
Aurora writes only REDO LOG RECORDS to storage nodes.
Storage nodes apply log to reconstruct actual data pages.
This is called "log-structured" storage.

Traditional DB replication:
Primary ──copy entire storage──► Replica
  (GBs to TBs of data copied when replica starts or re-syncs)

Aurora:
Writer Instance ──writes log records──► Distributed Storage
Reader Instance ──reads from──► Same Distributed Storage
  (no data copying — all instances share identical storage)

Physical layout:
├── Storage divided into Protection Groups (PGs)
│   Each PG = 10GB segment of data
├── Each PG has 6 copies: 2 copies in each of 3 AZs
├── Write quorum: 4/6 must acknowledge before write succeeds
├── Read quorum: 3/6 must respond
└── Storage nodes auto-repair any diverged copies

Durability model:
├── Lose 2 copies (1 AZ)   → still write (4 remaining)
├── Lose 3 copies (1.5 AZs) → still read (3 remaining)
└── Complete AZ failure     → still fully operational
```

```
Aurora I/O pricing:
├── Storage:          $0.10/GB-month
├── I/O (standard):   $0.20/million requests
└── I/O-Optimized:    $0.225/GB-month storage, NO I/O charges
    Use I/O-Optimized when I/O costs exceed 25% of total Aurora bill
```

🧩 Aurora Fast Failover

```
Why Aurora failover is faster than RDS Multi-AZ:

RDS Multi-AZ failover:
├── Primary fails
├── Standby promoted to primary
├── Standby must complete crash recovery (replay uncommitted log)
├── DNS CNAME updated
└── Total: 60-120 seconds

Aurora failover:
├── Writer instance fails
├── Replica promoted to writer
│   (shares same storage — no data to sync, state is immediate)
├── Crash recovery: minimal (just replay small local cache)
├── DNS CNAME updated
└── Total: typically < 30 seconds (often < 10 seconds)

Aurora failover tiers:
├── Tier 0:   fastest promotion priority
├── Tier 1-15: progressively lower priority
└── ⚠ Assign lower tier numbers to replicas with more capacity
   (they become primary faster — better for the workload)

Aurora Blue/Green Deployments:
├── Create a "green" environment (new version or schema)
├── Green replicates from blue (production remains live)
├── Test on green
├── Switch: traffic cutover in < 1 minute
├── Can switch back immediately if issues found
└── Use for: major version upgrades, schema changes
```

🧩 Aurora Parallel Query

```
Aurora Parallel Query (Aurora MySQL-compatible):
Pushes query processing DOWN to the storage layer instead of
pulling all data up to the compute layer.

Traditional analytical query:
SELECT COUNT(*), SUM(amount) FROM orders WHERE date > '2024-01-01'
```

```
Without Parallel Query:
→ Storage sends ALL 100M rows to compute layer
→ Compute filters and aggregates
→ Massive network transfer between storage and compute

With Parallel Query:
→ Storage nodes each process their local portion
→ Only AGGREGATED results sent to compute layer
→ Tiny network transfer (just the aggregates)

Benefits:
├── Analytical queries 2-100x faster on large tables
├── Less I/O load on compute instance
└── OLTP queries NOT affected (normal path for row lookups)

Requirements:
├── Aurora MySQL 2.09+ (MySQL 5.7 compatible)
└── Supported instance types (r3, r4, r5 series)

When to use Parallel Query vs alternatives:
├── Parallel Query: mixed OLTP + occasional analytics on same cluster
├── Redshift:       dedicated analytics warehouse (heavy, complex SQL)
└── Aurora → S3 export → Athena: serverless analytics

⚠ Parallel Query may increase I/O costs (scanning more storage nodes).
   Not worth it for small tables or simple indexed queries.
   Best for large table full-scan analytical queries.
```

💬 Short Crisp Interview Answer

"Aurora's storage architecture stores only redo log records in a distributed storage system with 6 copies across 3 AZs. Write quorum requires 4/6, read quorum 3/6, surviving a full AZ failure. Because all instances share identical physical storage, failover is under 30 seconds — there's no data to synchronize, the new writer simply inherits the storage state. Parallel Query pushes analytical computations down into the storage layer itself, so instead of pulling millions of rows to compute, only aggregated results come back — making large-table analytics 2-100x faster without needing a separate warehouse."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Aurora I/O costs accumulate	High-read workloads pay per million I/O ops. Switch to I/O-Optimized when I/O > 25% of bill
Parallel Query not universal	Only helps large table full-scans. Indexed lookups, small tables — no benefit
Failover tier defaults	All replicas default to tier 15. Assign priorities or failover is non-deterministic

Gotcha	Detail
Aurora failover ≠ instant	< 30 seconds, but still requires app reconnection. Handle reconnection gracefully in code

---

6.8 DynamoDB — Hot Partition Problem, Adaptive Capacity, Design Patterns

---

🟢 What It Is in Simple Terms

The hardest part of DynamoDB is data modeling — specifically, avoiding hot partitions where all traffic concentrates on one physical partition. This section covers identifying and fixing hot partitions and the key patterns for efficient DynamoDB schemas.

🧩 Hot Partition Problem

```
DynamoDB partition limits:
Each partition handles:
├── 3,000 RCU (reads per second)
├── 1,000 WCU (writes per second)
└── 10 GB of data

Hot partition = one partition receives disproportionate traffic

Example scenario:
Table: UserPosts
PK: userId, SK: postId

Celebrity user "MrBeast" has 200M followers.
When MrBeast posts:
→ Millions of reads on PK = "MrBeast"
→ ALL routed to single partition
→ That partition's 3,000 RCU exhausted instantly
→ Other users' reads on that same partition also throttled!

Common hot partition causes:
├── Celebrity/popular item (one user has massive traffic)
├── Sequential keys: date-based PKs (2024-01-15)
│   → All current writes go to latest date partition
├── Low cardinality PK (status = "ACTIVE" / "INACTIVE")
│   → Only 2 partitions, all traffic concentrated on 2
└── Viral product (one SKU vs millions)
```

🧩 Adaptive Capacity

```
DynamoDB Adaptive Capacity (automatic):
├── Automatically redistributes capacity to hot partitions
├── Happens within minutes of detecting imbalance
├── Can temporarily give a partition MORE than its fair share
└── Does NOT increase total table capacity — just redistributes

Limitations:
├── If total table throughput is exhausted → still throttled
└── Handles unexpected bursts; sustained hot partitions need redesign

Instantaneous Adaptive Capacity (newer feature):
├── Shifts capacity INSTANTLY (not minutes)
└── Better handles sudden viral traffic spikes

⚠ Adaptive Capacity is NOT a substitute for good data modeling.
   It handles unexpected bursts but cannot fix fundamental
   schema design problems with low-cardinality partition keys.

Adaptive Capacity is DynamoDB's automatic mitigation — it redistributes
capacity to hot partitions within minutes, but it's not instant. Errors
still occur during rebalancing. It's not a substitute for good key design.

On-Demand mode does not eliminate hot partitions — it raises the ceiling
but a single continuously hot key can still throttle even in On-Demand
mode.
```

🧩 Hot Partition Solutions

```
Solution 1: Write Sharding (for write-heavy hot keys)
   Instead of PK = "status" (only 2 values):
   Use PK = "status#shard" where shard = random 1-10

   Write: randomly choose shard → "ACTIVE#1", "ACTIVE#7"...
   Read:  scatter-gather — query all 10 shards in parallel,
          merge results

   Tradeoff: reads are more complex (parallel queries + merge)
   Use when: writes dominate and reads can tolerate scatter-gather

Solution 2: Partition Key with Random Suffix
   Celebrity account PK: "user#MrBeast#3" (random 0-9)
   → Writes distributed across 10 partitions
   → Reads: must query all 10 suffixes + merge results

Solution 3: Time-based keys with sharding
   Instead of: PK = "2024-01-15" (sequential → hot)
   Use: PK = "2024-01-15#shard" (shard = random 0-99)
   → 100 partitions for the same date
```

```
Solution 4: Avoid low-cardinality partition keys entirely
   Status field? Move to SK or filter expression, not PK.
   Keep PK high cardinality (userId, orderId, requestId).

Solution 5: Cache hot items
   Use DAX in front of DynamoDB.
   Celebrity profiles served from cache — not DynamoDB.
   Only writes (and cache misses) hit DynamoDB.
```

🧩 DynamoDB Design Patterns

```
Pattern 1: Single-Table Design
Put multiple entity types in ONE table.
Avoid joins by co-locating related data together.

┌──────────────────┬──────────────────────┬──────────────────┐
│ PK               │ SK                   │ Data             │
├──────────────────┼──────────────────────┼──────────────────┤
│ USER#123         │ PROFILE              │ {name, email}    │
│ USER#123         │ ORDER#2024-001       │ {total, status}  │
│ USER#123         │ ORDER#2024-002       │ {total, status}  │
│ PRODUCT#456      │ DETAILS              │ {name, price}    │
│ PRODUCT#456      │ INVENTORY            │ {stock, location}│
└──────────────────┴──────────────────────┴──────────────────┘

Access patterns served:
- Get user profile:     PK=USER#123, SK=PROFILE
- Get all user orders:  PK=USER#123, SK begins_with ORDER#
- Get specific order:   PK=USER#123, SK=ORDER#2024-001
- Get product details:  PK=PRODUCT#456, SK=DETAILS

Pattern 2: Adjacency List (Graph / Many-to-Many)
Model many-to-many relationships in one table:

PK=BILL#123, SK=BILL#123 → invoice data
PK=BILL#123, SK=PROD#456 → bill contains product 456
PK=BILL#123, SK=PROD#789 → bill contains product 789
PK=PROD#456, SK=PROD#456 → product data
PK=PROD#456, SK=BILL#123 → product appears in bill 123

GSI: PK=SK, SK=PK → reverse the relationship
Access: all bills containing product 456 → GSI query PK=PROD#456

Pattern 3: Composite Sort Key for hierarchy
PK = "ORG#acme"
SK = "DEPT#engineering"
SK = "DEPT#engineering#TEAM#platform"
SK = "DEPT#engineering#TEAM#platform#USER#john"

Queries:
```

```
  - All departments:              SK begins_with "DEPT#"
  - All teams in engineering:    SK begins_with "DEPT#engineering#TEAM#"
  - All users in platform team:  SK begins_with
  "DEPT#engineering#TEAM#platform#"

  Pattern 4: Overloaded GSI
  Use ONE GSI to serve MULTIPLE access patterns.

  GSI1PK and GSI1SK attributes overloaded per entity type:
  USER items:    GSI1PK="ORG#acme",        GSI1SK="USER#john"
  ORDER items:   GSI1PK="STATUS#SHIPPED",  GSI1SK="2024-01-15"
  PRODUCT items: GSI1PK="CAT#electronics", GSI1SK="PROD#456"

  ONE GSI serves: all users in org, all shipped orders by date,
                  all products by category.

  Pattern 5: Version tracking with sort key
  PK = "DOC#contract123"
  SK = "v001", "v002", "v003" → document versions
  SK = "LATEST" → current version (duplicate for fast access)

  Always-consistent "get latest version" without scanning.
```

💬 Short Crisp Interview Answer

"Hot partitions in DynamoDB occur when traffic concentrates on one physical partition — typically due to celebrity users, sequential date-based keys, or low-cardinality partition keys. Each partition handles 3,000 RCU and 1,000 WCU. Adaptive Capacity automatically redistributes capacity to hot partitions within minutes but cannot increase total table throughput and isn't a substitute for proper design. Solutions include write sharding with a random suffix and scatter-gather reads, using high-cardinality keys, and caching hot items in DAX. The single-table design pattern co-locates related entities in one table using PK/SK prefixes with an overloaded GSI serving multiple access patterns. Access patterns must drive design — retrofitting DynamoDB schemas is expensive."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Adaptive Capacity has limits	Can't help if total table capacity is exhausted — only redistributes
Write sharding = read complexity	Scatter-gather reads across shards. Decide if the tradeoff is worth it
Sequential keys = hot partition	Date-based PKs route all writes to one partition
Low cardinality PK is fatal	status=ACTIVE/INACTIVE → only 2 partitions for entire table
Single-table design is hard to undo	Wrong PK/SK choices require expensive table rebuilds and data migrations

Gotcha	Detail
GSI throttling affects base table	GSI write capacity throttled → base table writes also throttled

---

6.9 RDS Proxy — Connection Pooling, Use with Lambda

🟢 What It Is in Simple Terms

RDS Proxy sits between your application and RDS database, maintaining a pool of warm database connections. Instead of each application instance opening its own connection, they share the proxy's connection pool. This is critical for Lambda functions which create new database connections on every cold start.

🔍 Why It Exists / What Problem It Solves

```
The Problem — Lambda + RDS = Connection Exhaustion:

Without RDS Proxy:
1,000 Lambda invocations
→ 1,000 database connections opened simultaneously
→ RDS max_connections (db.t3.medium): ~330
→ Connections refused → Lambda failures
→ "too many connections" errors

Connection exhaustion symptoms
Connection exhaustion symptoms in CloudWatch: DatabaseConnections near
max_connections + CPUUtilization high + ReadLatency spiking — even with
normal IOPS. The CPU spike is caused by MySQL thread context switching
overhead, not query complexity. This pattern is distinct from a slow query
problem (where ReadIOPS or WriteIOPS would also be elevated).

PostgreSQL max_connections by instance size:
db.t3.micro:   ~77
db.t3.small:   ~150
db.t3.medium:  ~330
db.r5.large:   ~680
db.r5.xlarge:  ~1,350

With RDS Proxy:
1,000 Lambda invocations → RDS Proxy
→ Proxy maintains a fixed pool (e.g., 20 DB connections)
→ Lambdas borrow connections from pool
→ Database sees only 20 connections regardless of Lambda count
→ No connection exhaustion
```

⚙️ How RDS Proxy Works

```
Connection Multiplexing:
├── Many app connections → few actual DB connections
├── Multiplexing works when: no active transactions,
│   no SET statements, no prepared statements
└── Pinning: transaction/session state requires dedicated connection

⚠ Operations that PREVENT multiplexing (pin the connection):
   - SET commands that change session state
   - Prepared statements (unless server-side)
   - Stored procedures with multiple result sets

Pinned connections reduce the pooling benefit.
Avoid SET statements and be careful with transactions.
```

```aws-cli
# Create RDS Proxy
aws rds create-db-proxy \
  --db-proxy-name prod-db-proxy \
  --engine-family POSTGRESQL \
  --auth '[{
    "AuthScheme": "SECRETS",
    "SecretArn": "arn:aws:secretsmanager:...:secret:db-creds",
    "IAMAuth": "REQUIRED"
  }]' \
  --role-arn arn:aws:iam::123:role/rds-proxy-role \
  --vpc-subnet-ids subnet-a subnet-b subnet-c \
  --vpc-security-group-ids sg-proxy

# Register target (link proxy to RDS instance)
aws rds register-db-proxy-targets \
  --db-proxy-name prod-db-proxy \
  --db-instance-identifiers mydb-instance
```

🧩 Lambda + RDS Proxy Pattern

```python
import boto3
import psycopg2
import os

# Module-level connection (reused on warm invocations)
conn = None
PROXY_HOST = 'prod-db-proxy.proxy-xyz.us-east-1.rds.amazonaws.com'

def get_iam_token():
```

```
    """Generate IAM auth token for RDS Proxy (valid 15 minutes)"""
    rds = boto3.client('rds')
    return rds.generate_db_auth_token(
        DBHostname=PROXY_HOST,
        Port=5432,
        DBUsername='lambda_user',
        Region='us-east-1'
    )

def handler(event, context):
    global conn

    # Reuse connection if still open (warm Lambda environment)
    if conn is None or conn.closed:
        conn = psycopg2.connect(
            host=PROXY_HOST,
            port=5432,
            database='mydb',
            user='lambda_user',
            password=get_iam_token(),  # IAM token — no hardcoded password
            sslmode='require'
        )

    with conn.cursor() as cur:
        cur.execute("SELECT * FROM orders WHERE id = %s",
(event['orderId'],))
        return cur.fetchone()
```

```
RDS Proxy benefits for Lambda specifically:
├── Eliminates "too many connections" errors
├── Lambda cold start doesn't overwhelm database with new connections
├── Failover: proxy reconnects to new primary transparently
│   (Lambda doesn't need reconnection logic)
└── IAM auth: no credentials in code or environment variables

⚠ RDS Proxy costs:
   $0.015 per vCPU per hour of the proxied DB instance.
   db.r5.large (2 vCPU): 2 × $0.015 = $0.03/hr = ~$22/month
   Typically well worth it for Lambda workloads.

⚠ RDS Proxy is VPC-internal.
   Lambda must be in the same VPC as the proxy.
   (Lambda VPC deployment has its own considerations.)
```

💬 Short Crisp Interview Answer

"RDS Proxy is a managed connection pooler that sits between your applications and RDS, maintaining a warm pool of database connections. Without it, Lambda functions are a database killer — 1,000 concurrent invocations each open their own connection, immediately exhausting RDS connection limits. Proxy

multiplexes thousands of application connections onto a small pool of real database connections. It also handles RDS failover more gracefully — the proxy reconnects to the new primary transparently. RDS Proxy also enables IAM authentication, so Lambda connects using its execution role rather than hardcoded credentials. The key gotcha is connection pinning: SET statements or transactions prevent multiplexing and reduce the pooling benefit."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Connection pinning	SET statements, certain prepared statements pin connections — reduces pooling benefit
Proxy adds latency	~1ms additional per query. Acceptable for most use cases
Only MySQL and PostgreSQL	No Oracle, no SQL Server support
VPC only	Lambda must be in same VPC as proxy
IAM token expiry	IAM auth tokens valid 15 minutes. Must refresh before expiry
Proxy cost	~$22/month per 2 vCPU DB. Small price for eliminating connection exhaustion

---

6.10 Redshift — Architecture, Distribution Styles, Sort Keys

🟢 What It Is in Simple Terms

Redshift is AWS's fully managed data warehouse — a petabyte-scale columnar database optimized for analytical queries (aggregations over millions of rows) rather than transactional queries (point lookups on individual rows).

🔍 Why It Exists / What Problem It Solves

OLTP databases (RDS, Aurora) are optimized for transactions: insert one row, look up one customer. Analytics queries (sum all sales by region for the last year) touch millions or billions of rows — fundamentally different access patterns. Redshift is built for massive parallel processing of analytical queries.

⚙️ Redshift Architecture

```
Redshift Cluster Architecture:

┌──────────────────────────────────────────────────────────────┐
│                      Leader Node                              │
│  ├── Receives SQL queries from clients                        │
│  ├── Creates query execution plan                             │
│  ├── Distributes compiled code to compute nodes               │
│  ├── Aggregates results from compute nodes                    │
│  └── Returns final result to client                          │
└──────────────────────────────────────────────────────────────┘
                        │ distributes work
        ┌───────────────┼───────────────┐
        ▼               ▼               ▼

┌──────────┐    ┌──────────┐    ┌──────────┐
│ Compute  │    │ Compute  │    │ Compute  │
│ Node 1   │    │ Node 2   │    │ Node 3   │
│ Slice 1  │    │ Slice 3  │    │ Slice 5  │
│ Slice 2  │    │ Slice 4  │    │ Slice 6  │
│(data +   │    │(data +   │    │(data +   │
│ compute) │    │ compute) │    │ compute) │
└──────────┘    └──────────┘    └──────────┘

Slices:
├── Each compute node has multiple slices (CPUs)
├── Each slice holds a portion of the total data
└── Slices on all nodes process queries in parallel simultaneously

Columnar storage:
Traditional row storage — all columns for each row are adjacent:
  Row 1: [id=1, name="Alice", amount=100, date="2024-01"]
  Row 2: [id=2, name="Bob",   amount=200, date="2024-01"]

Columnar storage — all values of each column are adjacent:
  id:     [1, 2, 3, 4, ...]
  name:   ["Alice", "Bob", "Carol", ...]
  amount: [100, 200, 300, ...]
  date:   ["2024-01", "2024-01", ...]

Query: SELECT SUM(amount) FROM sales
  Row storage:    reads ALL columns for all rows → wastes I/O
  Columnar:       reads ONLY the amount column → fraction of I/O

Columnar benefits:
├── Column pruning: only read needed columns
├── Compression: same data type → high compression ratios
│   amount column compresses much better than mixed-type rows
└── SIMD: vectorized operations on batches of column values

Redshift node types:
├── RA3: managed storage (S3-backed), compute and storage scale
│         independently — best for most use cases
└── DC2: local SSD storage, compute and storage coupled together
          best for fixed, hot data requiring maximum performance
```

```
Redshift Serverless:
├── No cluster to manage
├── Scales compute automatically based on workload
├── Pay per RPU (Redshift Processing Unit) per second used
└── Good for: variable analytical workloads, dev/test
```

🧩 Distribution Styles

```
Distribution Style = how Redshift places rows across slices.
CRITICAL for query performance — bad distribution = slow joins.

When joining two large tables:
├── Matching rows on DIFFERENT slices → must shuffle data
│   across nodes (expensive: network transfer + waits)
└── Matching rows on SAME slice → join is LOCAL (fast, free)

Four distribution styles:

1. AUTO (default):
   └── Redshift chooses ALL or KEY based on table size.
       Small tables → ALL, large tables → KEY.
       Good starting point for new tables.

2. EVEN:
   ├── Rows distributed round-robin across all slices
   ├── Even row distribution (no skew)
   ├── Good when: table not used in frequent joins
   └── Bad when: frequently joined — matching rows on different nodes

3. KEY (best for large tables in frequent joins):
   ├── Rows with SAME distribution key value → go to SAME slice
   ├── If Table A and Table B both distributed on customer_id:
   │   → All rows for customer 123 in BOTH tables on same slice
   │   → Join is LOCAL (no data movement) → fast!
   └── ⚠ Key skew: one customer with millions of rows
       → one slice gets disproportionate data → hotspot

4. ALL:
   ├── ENTIRE table copied to EVERY node
   ├── All joins are always local (matching row always on same node)
   ├── Good for: small dimension tables (< 100K rows)
   └── Bad for: large tables (copies waste storage + slow updates)
       Updating ALL table = must update every copy on every node

Example join optimization:
Large fact table (sales):         DISTSTYLE KEY ON customer_id
Large dimension table (customers): DISTSTYLE KEY ON customer_id
Small dimension table (regions):   DISTSTYLE ALL
```

```
Query:
SELECT * FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
JOIN regions r   ON c.region_id   = r.region_id

Result:
- sales JOIN customers → LOCAL (same slice, same customer_id)
- JOIN regions         → LOCAL (ALL table on every node)
→ Zero data movement → maximum query speed!
```

🧩 Sort Keys

```
Sort Key = physical order of rows on disk
           (like an index, but for sequential range scans)

How sort keys help — Zone Maps:
Each 1MB disk block records its min and max sort key values.
If query filter is OUTSIDE a block's min-max range → SKIP the block.

Without sort key:
SELECT SUM(amount) FROM sales WHERE date BETWEEN '2024-01' AND '2024-03'
→ Must scan ALL blocks (no way to skip any)

With sort key on date:
→ Block zone map: min=2024-01, max=2024-01 → INCLUDE
→ Block zone map: min=2023-01, max=2023-12 → SKIP
→ Only relevant blocks scanned → potentially 100x less I/O

Sort key types:

1. Compound Sort Key (default, recommended for most cases):
   ├── All columns matter but only in left-to-right prefix order
   ├── Put most selective / frequently filtered column FIRST
   └── Example: SORTKEY (year, month, day)
       Query on year+month:  effective (uses zone maps)
       Query on month only:  NOT effective (skips year prefix)

2. Interleaved Sort Key:
   ├── Gives equal weight to ALL columns in the sort key
   ├── Effective for queries filtering on ANY column subset
   ├── Example: INTERLEAVED SORTKEY (region, product_category, date)
   │   Query on any single column → still uses zone maps
   └── ⚠ Slower VACUUM and LOAD operations (complex ordering)
       Generally prefer Compound unless many independent access patterns
```

```
-- Production table definition with distribution and sort keys
CREATE TABLE sales (
    sale_id      BIGINT         ENCODE zstd,
```

```
    customer_id  INTEGER        ENCODE zstd,
    product_id   INTEGER        ENCODE zstd,
    amount       DECIMAL(10,2)  ENCODE bytedict,
    sale_date    DATE           ENCODE RAW,
    region       VARCHAR(50)    ENCODE lzo
)
DISTSTYLE KEY
DISTKEY  (customer_id)
SORTKEY  (sale_date, region);
```

```
VACUUM and ANALYZE:
├── VACUUM:  reclaims space, re-sorts unsorted rows
│            (new inserts/updates land outside sort order)
├── ANALYZE: updates statistics for query planner
└── Both should run regularly (Redshift AUTO VACUUM available)
```

🧩 Redshift Spectrum & Federated Query

```
Redshift Spectrum:
├── Query data IN S3 directly (without loading into Redshift)
├── Extends Redshift queries to exabytes of S3 data
├── Uses separate Spectrum nodes (not your cluster compute)
└── Best format: Parquet (columnar, compressed — minimize scan cost)
    Cost: $5 per TB of S3 data scanned
```

```sql
-- Create external schema pointing to Glue Data Catalog
CREATE EXTERNAL SCHEMA spectrum
FROM DATA CATALOG
DATABASE 'my_glue_db'
IAM_ROLE 'arn:aws:iam::123:role/redshift-spectrum-role';

-- Query spans hot (Redshift) and cold (S3) data transparently
SELECT year, SUM(amount)
FROM recent_sales                        -- local Redshift table
UNION ALL
SELECT year, SUM(amount)
FROM spectrum.historical_sales           -- external S3 data
GROUP BY year;
```

```
Federated Query:
├── Query data in live RDS/Aurora FROM Redshift
└── Joins across data warehouse + transactional DB without ETL
```

💬 Short Crisp Interview Answer

"Redshift is AWS's managed columnar data warehouse for petabyte-scale analytics. The architecture has a leader node that plans and distributes queries, and compute nodes with slices that process data in parallel. Columnar storage means queries only read the columns they need, enabling high compression and fast analytical scans. Distribution style determines where rows physically live — KEY distribution co-locates matching rows from joined tables on the same slice to avoid costly data movement; ALL distribution copies small dimension tables to every node; EVEN is round-robin for non-join tables. Sort keys define physical row ordering so Redshift can skip entire disk blocks using zone maps — compound sort keys work for prefix filter patterns. Redshift Spectrum extends queries to S3 data without loading it, priced at $5/TB scanned."

⚠️ Tricky Edge Cases / Gotchas

Gotcha	Detail
Distribution key skew	KEY distribution with uneven data → one slice overwhelmed. Check slice row counts
ALL distribution update cost	Updating ALL table updates every copy on every node — expensive
Compound sort key prefix	Query on non-prefix columns ignores zone maps — use interleaved for multiple access patterns
VACUUM required	Unsorted rows from inserts/updates reduce zone map effectiveness. Run VACUUM regularly
Redshift not for OLTP	Point lookups are slow (full column scans). Use RDS/Aurora for transactional queries
Spectrum costs	$5/TB scanned. Use Parquet format and partition S3 data to minimize scan
Leader node bottleneck	All results aggregate through leader node. Large result sets = leader bottleneck

---

🔗 Category 6 — Full Connections Map

```
DATABASES connects to:

RDS
├── EC2/ECS/EKS/Lambda → application layer connecting to DB
├── RDS Proxy          → connection pooling (critical for Lambda)
├── Secrets Manager    → database credentials storage
├── KMS                → storage encryption key management
├── CloudWatch         → database metrics and alarms
├── S3                 → snapshot export (Parquet), restore
├── VPC                → DB subnet groups, Security Groups
```

```
  ├── Multi-AZ           → standby in different AZ (HA)
  ├── Read Replicas      → read scaling (async replication)
  └── Parameter Groups   → engine configuration tuning

  Aurora
  ├── All RDS connections (Aurora is RDS-compatible API)
  ├── Aurora Serverless  → auto-scaling compute
  ├── Global Database    → cross-region replication (< 1s lag)
  ├── S3                 → import/export, backup storage
  └── Parallel Query     → push-down analytics to storage layer

  DynamoDB
  ├── Lambda             → event source mapping via Streams
  ├── DAX                → in-memory cache (microsecond reads)
  ├── S3                 → export to S3 (Parquet), import
  ├── Streams            → change data capture to Lambda/Kinesis
  ├── EventBridge        → trigger from Streams via Lambda
  └── TTL                → auto-expire items (no IAM needed)

  ElastiCache
  ├── EC2/ECS/EKS/Lambda → applications using Redis/Memcached
  ├── VPC                → private subnet deployment (VPC only)
  ├── Secrets Manager    → Redis AUTH password storage
  ├── CloudWatch         → cache hit rate, evictions, memory
  └── KMS                → at-rest encryption keys

  Redshift
  ├── S3                 → COPY command (load data), Spectrum
  ├── Glue               → ETL from various sources to Redshift
  ├── QuickSight         → BI visualization on Redshift data
  ├── Lambda             → trigger data loads, transformations
  ├── IAM                → cluster access, Spectrum S3 access
  └── VPC                → private cluster deployment
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
RDS Multi-AZ replication	Synchronous. Standby NOT readable. 60-120s failover
RDS Read Replica replication	Asynchronous. Readable. Own endpoint. Up to 5 (15 Aurora)
RDS PITR	Any second within retention. Restores to NEW instance
Manual snapshot retention	Forever until manually deleted
Automated backup retention	1-35 days (default 7)
DynamoDB item size limit	400KB max per item

Topic	Key Fact
DynamoDB partition limits	3,000 RCU + 1,000 WCU + 10GB per partition
DynamoDB RCU (strong)	1 RCU = 4KB strongly consistent (8KB item = 2 RCU)
DynamoDB RCU (eventual)	1 RCU = 8KB eventually consistent
DynamoDB WCU	1 WCU = 1KB (3.5KB item = 4 WCU — always round up)
LSI when to create	At table creation ONLY. Cannot add to existing table
GSI when to create	Anytime — before or after table creation
LSI consistency	Strong or eventual (your choice). GSI = eventual only
Aurora replication lag	Sub-10ms (vs seconds for RDS)
Aurora failover time	< 30 seconds (vs 60-120s for RDS Multi-AZ)
Aurora storage copies	6 copies across 3 AZs. Write quorum 4/6, read quorum 3/6
Aurora max storage	128TB auto-growing in 10GB increments
Aurora max read replicas	15 (vs 5 for standard RDS)
Aurora Serverless v2 min	0.5 ACU minimum — does NOT scale to zero
Global Database replication	< 1 second lag. Up to 5 secondary regions
Global Database failover	Manual — no automatic promotion yet
DynamoDB On-Demand cost	~5x more expensive per request than provisioned at high sustained load
Auto-scaling reactivity	60-90 seconds to respond — bursts can exhaust capacity first
DAX consistency	Eventually consistent reads from cache. Strongly consistent = bypass DAX
DynamoDB Streams retention	24 hours — must process within window or records lost
TTL accuracy	Within 48 hours — NOT instantaneous
TTL expired items	Still returned by Query/Scan until actually deleted. Filter in app code
Redis vs Memcached	Redis = persistence + replication + rich types. Memcached = simple pure cache
Redis Cluster Mode	Shards across nodes. Multi-key ops MUST use hash tags for same slot
RDS Proxy use case	Lambda + RDS = connection exhaustion. Proxy pools connections
RDS Proxy connection pinning	SET statements prevent multiplexing — reduces pooling benefit
RDS Proxy only supports	MySQL and PostgreSQL (not Oracle, not SQL Server)

Topic	Key Fact
Redshift columnar	Only reads needed columns — massive I/O savings for analytics
Redshift DISTSTYLE KEY	Co-locates joined rows on same slice — eliminates data movement
Redshift DISTSTYLE ALL	Copies entire small table to every node — all joins are local
Redshift sort key	Zone maps skip irrelevant blocks — compound key uses prefix filtering
Hot partition cause	Low cardinality PK, sequential date keys, celebrity/viral items
Hot partition fix	Write sharding (random suffix + scatter-gather), DAX caching, redesign
Adaptive Capacity	Redistributes existing capacity to hot partitions — not a schema fix

Category 6: Databases — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation